



Figure 2: A plot showing observations and polynomial fits for two falling bodies

A polynomial can be represented by the coefficients as we have discussed earlier. It can be equally well represented by its roots. So, if you create an object of type `polynomial`, both representations are easily available to you, as follows:

```
>>> pf=np.poly1d(poly_coef)
>>> print 'The roots', pf.r
The roots [-0.69426607  0.58566055]
>>> print 'The coefficients', pf.c
The coefficients [-481.72027972 -52.31748252 195.86951049]
```

Multiple data sets

Typically, there will be multiple runs and you will need to analyse each set of data. In the following example, you have two runs of data—one for a stone and the second for a small plastic cap. Try the following code:

```
import numpy as np
set1 = [195.7,193.51,190.59,185.84,180.73,173.43,
        165.76,157.36,147.87,136.92,124.87]
set2 = [194.04,192.6,189.36,185.04,180.36,173.52,
        167.76,158.76,148.68,138.6,127.8]
y = np.array([set1,set2])
print 'Shape of the array', y.shape
t = np.linspace(0, 10.0/30, 11)
res = np.polyfit(t, y.transpose(), 2)
print 'The shape of the result', res.shape
print 'g = ', 2*res[0]
```

The array `y` is the list of data values for each set. However, `polyfit` requires the list to be grouped by corresponding values for all trial data sets. So, you need to take the transpose of the array `y` to fit the needs of the `polyfit` function. The result will be a set of three coefficients, one for each set of data. You get a pair of

values for `g`. The output will now be as follows:

```
Shape of the array (2, 11)
The shape of the result (3, 2)
g = [-963.44055944 -953.11888112]
```

You may need to do more experiments with plastic caps! But meanwhile, you can plot the two sets of data and the fitted lines as shown below:

```
import matplotlib.pyplot as plt
plt.plot(tset1,'o', label='Dataset 1')
plt.plot(tset2,'d', label='Dataset 2')
coeff = res.transpose()
plt.plot(tnppoly1d(coeff[0])t,'-', label='Fit 1')
plt.plot(tnppoly1d(coeff[1])t,'-', label='Fit 2')
plt.legend()
plt.show()
```

Pyplot will choose a different colour for each plot. The first set of points is drawn as small circles. The second set of points is drawn as little diamonds. The last two are continuous lines. Note that you needed to take the transpose of the result matrix for easy access to the coefficients of each polynomial. A legend will be useful in this case. See Figure 2.

You can find the details of all the options of plotting by using inbuilt help for Python:

```
>>> help(plt.plot)
```

What else

Polynomials are very useful. The integral of a polynomial is another polynomial. The derivative is another polynomial. Numpy includes functions, `polyint` and `polyder` for this purpose. The function `polymul` will return another polynomial that is the product of two polynomials. There is a wealth of functions available in Numpy, which makes working with polynomials in education and research a much easier task.

The Scipy module of Python includes everything of Numpy and some more. So, you can replace the import statement by '`import scipy as np`' and your code will still run fine. In case you are still hesitating to use Python for science, the article 'Python in Science: How long until a Nobel Prize? And Now For Something Completely Different' by Doug Hellmann, might interest you—www.doughellmann.com/articles/CompletelyDifferent-2007-11-science. **END** 

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com